

Python OOP.

Date:.....

Page:.....

1. What is OOP?

→ It is a programming approach that organizes code around object and class and helps to make program more organized, reusable and easier to extend.

2. Class - So, class is a blueprint or template for creating an objects. It defines attribute (data) and methods (behaviour)

Example:

```
class Student
```

```
pass
```

3. Object - An object is an instance of a class. Multiple objects can be created from the same class.

Example:

```
s1 = student()
```

```
s2 = student()
```

4. Constructor (`__init__()`)

→ The `__init__()` method is a special method that runs automatically when an object is created.

Example: `class Student`

```
def __init__(self, name, age):
```

```
    self.name = name
```

```
    self.age = age
```


5. Self → Self refers to the current object. It is used to access the object's attributes and method inside a class.

6. Methods → Methods are functions defined inside a class. It describes the action or behaviour that an object can perform.

Example:

```
class Student:
```

```
    def greet(self):
```

```
        print('Hello')
```

7) The Four Main Pillars of OOP

i) Encapsulation - It means combining data and methods inside a class and controlling how the data is accessed or modified.

ii) Inheritance - It allows a child class to reuse attributes and methods from a parent class.

Example:

```
class Animal:
```

```
    def speak(self):
```

```
        print('Animals speaks')
```

```
class Dog(Animal):
```

Dog inherits speak() from Animal

* Types of Inheritance

- i) Single Inheritance - One parent, one child.
- ii) Multiple Inheritance - multiple parents, one child.
- iii) Multilevel Inheritance - Inheritance through multiple levels
- iv) Hierarchical Inheritance - One parent, multiple child

8) Super() - It is used to access methods or the constructor of a parent class.

Example:

```
class Dog(Animal):
    def __init__(self):
        super().__init__()
```

9) Polymorphism: It means the same method or interface can behave differently depending on the object

Example: Class Dog:

```
def sound(self):
    print("Bark")
```

Class Cat:

```
def sound(self):
    print("Meow")
```

Both classes have sound(), but behave differently.

15) Method Overriding: When parent class and child class have same method this process is method overriding.

Super keyword is used to solve this problem and update accordingly in parent class.

16) Abstraction: It means hiding unnecessary implementation detail and giving access to only the essential features.

17) Access Modifiers/ Naming Convention

→ Python does not enforce java-style access modifiers in the same way, but naming conventions are used:

- Public: name
- Protected: _name
- Private style: __name

Notes:

1. Importance of OOP

- Code reusability

2. Getter and setter method are used to retrieve values, and update values with controlled logic respectively.

File Handling

1. What is file handling?

→ It is a process of creating, opening, reading, writing, updating and closing files using python.

Syntax:

```
file = open('file.txt', 'mode')
```

2. Important file modes:

- r → reads, opens an existing file for reading only
- w → writes; creates a file if doesn't exist or overwrites a file.
- a → appends; adds data to end of a file
- x → creates; creates a new file and fails if it is already exists.
- b → binary mode

Example:

```
file = open('data.txt', 'r')  
content = file.read()  
print(content)  
file.close()
```


3) Readline and Readlines

→ `readline()` reads one line at a time.

Example:

`line = file.readline()`

→ gives first line of a file.

→ `readlines()` reads the lines and return all lines as a list.

Example:

`lines = file.readlines()`

→ gives the whole content of a file.

4) Closing a file

→ It closes opened file and releases system resources

Example:

`file.close()`

5) Exception Handling with files

→ try and except are used to handle errors

Example:

try:

`with open('data.txt', 'r') as file:`

`print(file.read())`

except `FileNotFoundError`:

`print("File not found")`

If error is unknown we can use,
except `Exception` as e:
`print(e)`